Busca/Otimização Meta-heurística

Trabalho Computacional - AV3

Pedro Vieira da Silva Xavier
Graduando em Ciência da Computação
Universidade de Fortaleza
Fortaleza, Brasil
pedro.xavier@edu.unifor.br

Cainã Rios Osterno Rocha
Graduando em Ciência da Computação
Universidade de Fortaleza
Fortaleza, Brasil
cainarocha21@edu.unifor.br

***Abstract*—Este trabalho investiga a resolução de problemas de busca e otimização em domínios contínuos e discretos por meio de algoritmos meta-heurísticos. Foram utilizados Hill Climbing, Local Random Search (LRS) e Global Random Search (GRS) para otimizar funções contínuas sob restrições de domínio, buscando valores mínimos e máximos. Também foi tratado o Problema das 8 Rainhas, resolvido com Têmpera Simulada (Simulated Annealing) a partir de uma função de aptidão que maximiza o número de pares de rainhas sem conflito. O trabalho descreve a metodologia, os hiperparâmetros e os critérios de parada, além de apresentar a moda das soluções encontradas nos problemas contínuos e a busca das 92 soluções distintas do problema das 8 rainhas.**

***Index Terms*—Meta-heurísticas, Hill Climbing, Local Random Search, Global Random Search, Simulated Annealing, Problema das 8 Rainhas.**

I. Introdução

Problemas de otimização são comuns em computação e engenharia, especialmente em espaços de busca grandes ou contínuos. Algoritmos meta-heurísticos oferecem soluções eficientes em tempo viável, mesmo sem garantia de ótimo global. Este trabalho aplica e analisa os algoritmos Hill Climbing, Local Random Search, Global Random Search e Têmpera Simulada em problemas contínuos e no Problema das 8 Rainhas.

II. Metodologia

Este trabalho está dividido em Problema de Minimização/Maximização da Função Custo/Objetivo e Problema de Domínio Discreto. A linguagem utilizada para resolver esses problemas foi o Python, com o auxílio de bibliotecas como *numpy* para facilitar operações matemáticas e entre vetores; e *matplotlib* para visualizar os gráficos dos problemas.

A. Problema de Minimização/Maximização da Função Custo/Objetivo

Foram fornecidas 6 funções a serem minimizadas/maximizadas e seus respectivos domínios. Os algoritmos utilizados foram Hill Climbing, Local Random Search (LRS) e Global Random Search (GRS). Para cada algoritmo, foram criadas classes dedicadas a eles, onde nas instâncias dessas classes em cada problema pode-se alterar os hiperparâmetros com finalidade de encontrar o ótimo.

1) Problema 1:

$$f(x_1, x_2) = x_1^2 + x_2^2$$

$$x_1, x_2 \in [-100, 100]$$

TABLE I
HIPERPARÂMETROS DOS MODELOS - PROBLEMA 1

Modelo	ϵ	**max_vizinhos**	σ	**max_it**	**t**
Hill Climbing	0.4	20	–	1000	100
Local Random Search	–	–	0.25	1000	100
Global Random Search	–	–	–	1000	1000

2) Problema 2:

$$f(x_1, x_2) = e^{-(x_1^2+x_2^2)} + 2 \cdot e^{-((x_1-1.7)^2+(x_2-1.7)^2)}$$

$$x_1 \in [-2, 4], x_2 \in [-2, 5]$$

TABLE II
HIPERPARÂMETROS DOS MODELOS - PROBLEMA 2

Modelo	ϵ	**max_vizinhos**	σ	**max_it**	**t**
Hill Climbing	1.1	20	–	1000	100
Local Random Search	–	–	0.25	1000	100
Global Random Search	–	–	–	1000	1000

3) Problema 3:

$$f(x_1, x_2) = -20 \cdot e^{-0.2\sqrt{0.5(x_1^2+x_2^2)}} - e^{0.5(\cos(2\pi x_1)+\cos(2\pi x_2))} + 20 + e^1$$

$$x_1, x_2 \in [-8, 8]$$

TABLE III
HIPERPARÂMETROS DOS MODELOS - PROBLEMA 3

Modelo	ϵ	**max_vizinhos**	σ	**max_it**	**t**
Hill Climbing	0.9	20	–	1000	100
Local Random Search	–	–	0.35	1000	100
Global Random Search	–	–	–	1000	1000

4) Problema 4:

$$f(x_1, x_2) = (x_1^2 - 10 \cdot \cos(2\pi x_1) + 10) + (x_2^2 - 10 \cdot \cos(2\pi x_2) + 10)$$

$$x_1, x_2 \in [-5.12, 5.12]$$

TABLE IV
HIPERPARÂMETROS DOS MODELOS - PROBLEMA 4

Modelo	ϵ	**max_vizinhos**	σ	**max_it**	**t**
Hill Climbing	0.95	20	–	1000	100
Local Random Search	–	–	0.6	1000	100
Global Random Search	–	–	–	1000	1000

5) Problema 5:

$$f(x_1, x_2) = \frac{x_1 \cdot \cos(x_1)}{20} + 2 \cdot e^{-(x_1)^2 - (x_2 - 1)^2} + 0.01 \cdot x_1 \cdot x_2$$

$$x_1, x_2 \in [-10, 10]$$

TABLE V
HIPERPARÂMETROS DOS MODELOS - PROBLEMA 5

Modelo	ϵ	**max_vizinhos**	σ	**max_it**	**t**
Hill Climbing	0.5	20	–	1000	100
Local Random Search	–	–	2.5	1000	700
Global Random Search	–	–	–	1000	1000

6) Problema 6:

$$f(x_1, x_2) = x_1 \cdot \sin(4\pi x_1) - x_2 \cdot \sin(4\pi x_2 + \pi) + 1$$

$$x_1 \in [-1, 3], x_2 \in [-1, 3]$$

TABLE VI
HIPERPARÂMETROS DOS MODELOS - PROBLEMA 6

Modelo	ϵ	**max_vizinhos**	σ	**max_it**	**t**
Hill Climbing	1.3	30	–	1000	300
Local Random Search	–	–	0.5	1000	300
Global Random Search	–	–	–	1000	1000

Vale ressaltar que os algoritmos foram executados em 100 rodadas e a métrica utilizada para encontrar o valor ótimo foi a moda (valor mais frequente). Sobre a moda, os resultados dos algoritmos não são números com poucas casas decimais, por se tratarem de números aleatórios, então utilizei uma abordagem de arredondar os resultados do Hill Climbing para três casas decimais, do LRS para duas, e para o GRS, apenas uma casa decimal.

É importante citar que a escolha dos hiperparâmetros foi feita por vários testes destes até encontrar uma solução ótima consistente. Sobre o hiperparâmetro *t* do GRS, eu removi ele colocando o mesmo valor do número máximo de iterações, pois não faz sentido eu parar esse algoritmo previamente, uma vez que ele possui visibilidade a todo espaço de estados e pode selecionar qualquer valor dele.

B. Problema de Domínio Discreto

A segunda parte do trabalho tratou da otimização combinatória de um problema discreto, o Problema das 8 Rainhas, que exige o posicionamento de 8 rainhas em um tabuleiro 8×8 de forma que nenhuma se ataque. Este é um problema de maximização com solução ótima conhecida (28 pares não atacantes) e foi abordado utilizando o algoritmo de Têmpera Simulada (Simulated Annealing).

A implementação do SA foi projetada para minimizar o número de conflitos ($h(x)$), sendo o ótimo global alcançado quando $h(x) = 0$.

1) Hiperparâmetros: O algoritmo também tem uma classe dedicada a ele, *SA8Rainhas*, permitindo controlar precisamente seus hiperparâmetros. Que estes, por sua vez, são:

- Número máximo de iterações: $max_it = 5000$
- Temperatura inicial (T_0): $T = 10$
- Resfriamento da temperatura: $\alpha = 0.99$
- Ruído (não foi utilizado): $\sigma = 1$

2) Busca: O experimento foi executado em duas fases principais: a busca por uma única solução e a busca exaustiva por 92 soluções diferentes. A busca exaustiva exigiu que o algoritmo SA fosse executado repetidas vezes com um novo estado inicial aleatório em cada run, até que o conjunto de 92 soluções únicas para o Problema das 8 Rainhas fosse completo.

3) Custo Computacional: Para analisar a eficiência do algoritmo SA, foram coletadas e calculadas as seguintes métricas a partir das execuções que resultaram em uma solução ótima ($h(x) = 0$):

- Tempo Total: tempo total (em segundos) consumido desde o início da busca até a identificação da 92ª solução única.
- Total de Execuções (runs): número total de vezes que o SA foi instanciado e executado.
- Iterações Totais: soma de todas as iterações consumidas exclusivamente pelas execuções que resultaram em uma solução ótima.
- Iterações Médias por Solução: média das iterações necessárias para encontrar uma solução ótima.
- Tempo Médio por Solução: média do tempo de execução dos runs que encontraram uma solução ótima.

Estes dados permitiram quantificar o custo computacional associado à exploração completa do espaço de soluções ótimas com o Têmpera Simulada.

III. RESULTADOS

A. Problema de Minimização/Maximização da Função Custo/Objetivo

1) Problema 1: Moda e gráficos

TABLE VII
TABELA DA MODA DOS MODELOS DE BUSCA - PROBLEMA 1

Modelo	Moda	Qtd. de vezes
Hill Climbing	np.float64(0.003), np.float64(-0.002)	3
LRS	np.float64(-0.01), np.float64(0.02)	4
GRS	np.float64(-0.0), np.float64(-0.1)	2

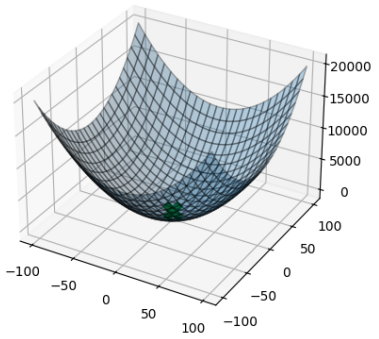

Fig. 1. Gráfico da solução do problema 1 (Hill Climbing)

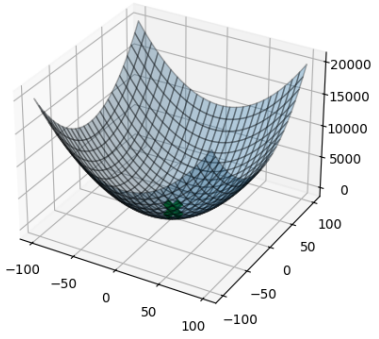

Fig. 2. Gráfico da solução do problema 1 (Local Random Search)

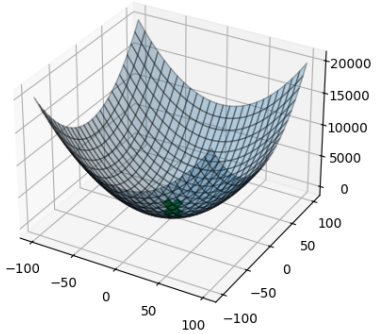

Fig. 3. Gráfico da solução do problema 1 (Global Random Search)

2) *Problema 2:* Moda e gráficos

TABLE VIII
TABELA DA MODA DOS MODELOS DE BUSCA - PROBLEMA 2

Modelo	Moda	Qtd. de vezes
Hill Climbing	np.float64(1.7), np.float64(1.698)	2
LRS	np.float64(1.71), np.float64(1.71)	5
GRS	np.float64(1.7), np.float64(1.7)	22

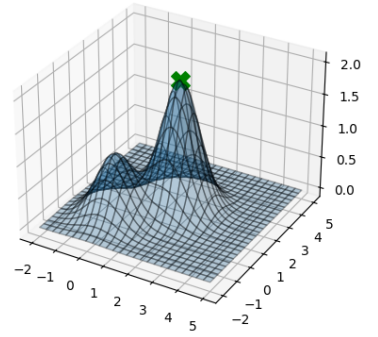

Fig. 4. Gráfico da solução do problema 2 (Hill Climbing)

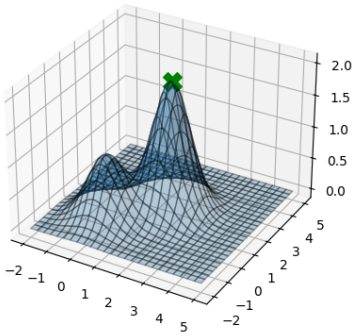

Fig. 5. Gráfico da solução do problema 2 (Local Random Search)

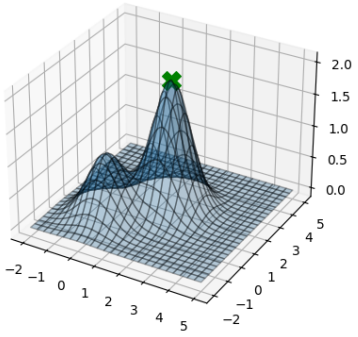

Fig. 6. Gráfico da solução do problema 2 (Global Random Search)

3) *Problema 3:* Moda e gráficos

TABLE IX
TABELA DA MODA DOS MODELOS DE BUSCA - PROBLEMA 3

Modelo	**Moda**	**Qtd. de vezes**
Hill Climbing	np.float64(-0.008), np.float64(0.012)	2
LRS	np.float64(0.02), np.float64(-0.0)	2
GRS	np.float64(-0.1), np.float64(-0.0)	7

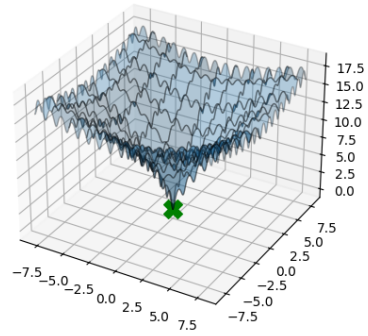

Fig. 7. Gráfico da solução do problema 3 (Hill Climbing)

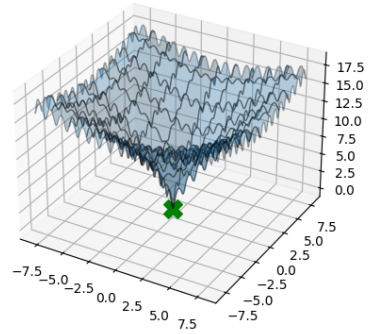

Fig. 8. Gráfico da solução do problema 3 (Local Random Search)

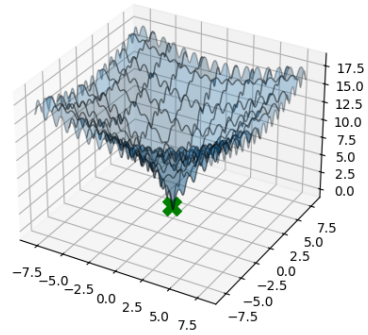

Fig. 9. Gráfico da solução do problema 3 (Global Random Search)

4) *Problema 4:* Moda e gráficos

TABLE X
TABELA DA MODA DOS MODELOS DE BUSCA - PROBLEMA 4

Modelo	Moda	Qtd. de vezes
Hill Climbing	np.float64(0.008), np.float64(0.001)	2
LRS	np.float64(0.02), np.float64(-0.02)	3
GRS	np.float64(-0.0), np.float64(-0.0)	11

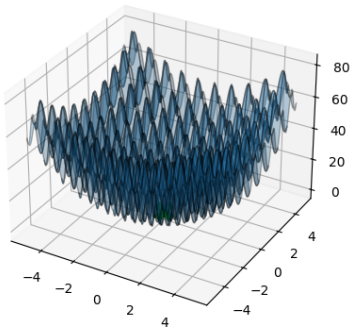

Fig. 10. Gráfico da solução do problema 4 (Hill Climbing)

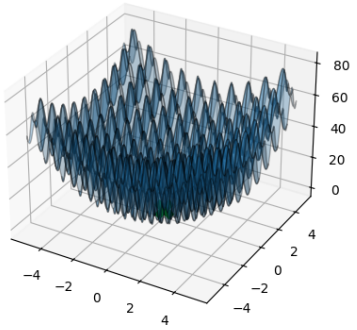

Fig. 11. Gráfico da solução do problema 4 (Local Random Search)

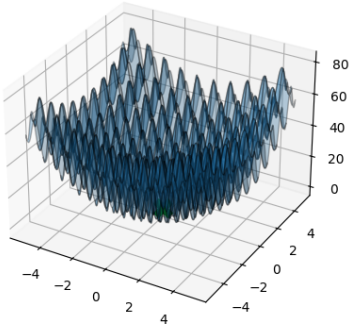

Fig. 12. Gráfico da solução do problema 4 (Global Random Search)

5) *Problema 5:* Moda e gráficos

TABLE XI
TABELA DA MODA DOS MODELOS DE BUSCA - PROBLEMA 5

Modelo	Moda	Qtd. de vezes
Hill Climbing	np.float64(-9.733), np.float64(-10.0)	80
LRS	np.float64(-9.73), np.float64(-10.0)	18
GRS	np.float64(0.0), np.float64(0.9)	6

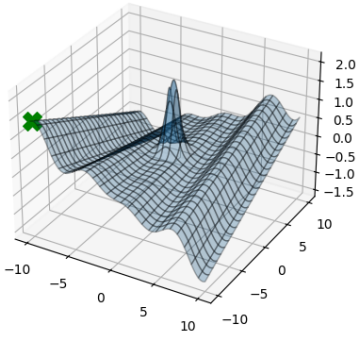

Fig. 13. Gráfico da solução do problema 5 (Hill Climbing)

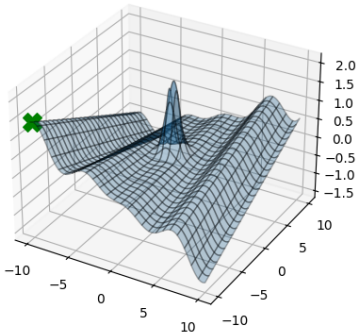

Fig. 14. Gráfico da solução do problema 5 (Local Random Search)

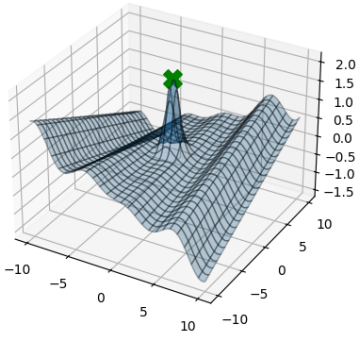

Fig. 15. Gráfico da solução do problema 5 (Global Random Search)

6) Problema 6: Moda e gráficos

TABLE XII
TABELA DA MODA DOS MODELOS DE BUSCA - PROBLEMA 6

Modelo	**Moda**	**Qtd. de vezes**
Hill Climbing	np.float64(2.62), np.float64(2.873)	2
LRS	np.float64(2.64), np.float64(2.86)	4
GRS	np.float64(2.6), np.float64(2.9)	32

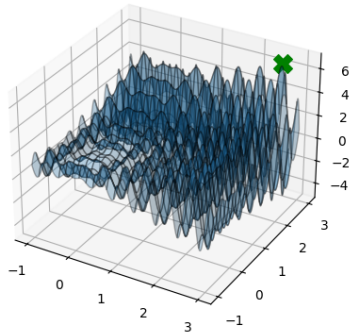

Fig. 16. Gráfico da solução do problema 6 (Hill Climbing)

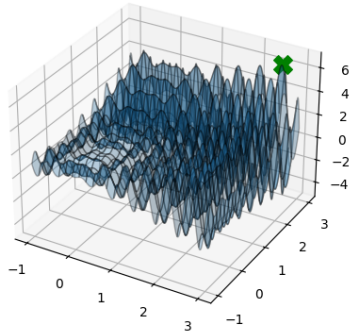

Fig. 17. Gráfico da solução do problema 6 (Local Random Search)

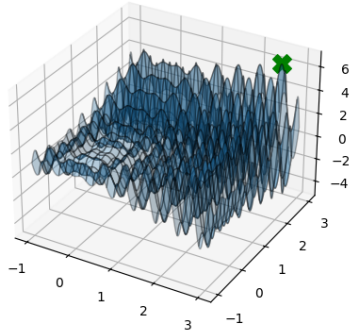

Fig. 18. Gráfico da solução do problema 6 (Global Random Search)

B. Problema de Domínio Discreto

1) Solução Única: Exemplo de solução única encontrada na execução do algoritmo.

TABLE XIII
EXEMPLO DE SOLUÇÃO ÚNICA NO PROBLEMA DAS 8 RAINHAS

Métrica	Valor
Solução encontrada	[3, 6, 2, 7, 5, 1, 8, 4]
Conflitos (f)	0
Iterações usadas	441
Tempo de execução	0.0286 s

2) 92 soluções distintas: Métricas das 92 soluções ao executar o algoritmo.

TABLE XIV
MÉTRICAS DAS 92 SOLUÇÕES NO PROBLEMA DAS 8 RAINHAS

Métrica	Valor
Solução únicas encontradas	92
Tempo total	12.40 s
Total de execuções do SA	577
Iterações totais consumidas	490875
Tempo médio por solução	0.0189 s
It. média por solução	5335.60
Runs por solução média	6.27

IV. CONCLUSÕES

O trabalho foi bem-sucedido na aplicação e análise de algoritmos meta-heurísticos tanto em problemas de otimização contínua quanto no domínio discreto.

Na primeira parte, a aplicação de **Hill Climbing** e **Local Random Search (LRS)** confirmou que a convergência para o ótimo é bem sensível ao ajuste dos hiperparâmetros (ϵ e σ). Tivemos que calibrar esses valores para cada um dos seis problemas, mas ainda assim, em alguns casos (como no Problema 5) ao aumentar a vizinhança o algoritmo ficou preso em um ótimo local. O **Global Random Search (GRS)** funcionou de forma consistente como um explorador global, e o uso da **moda** foi crucial para validar os resultados mais frequentes e confiáveis obtidos nas 100 rodadas, mas vale destacar que, para um espaço de estados com domínio grande, a busca global não é adequada.

Na segunda parte, o **Têmpera Simulada (Simulated Annealing - SA)** demonstrou ser muito eficaz para o **Problema das 8 Rainhas**. A escolha de uma temperatura inicial de $\mathbf{T_0 = 10}$ e um resfriamento lento ($\alpha = 0.99$) permitiu que o algoritmo escapasse de ótimos locais, garantindo a busca por soluções ótimas ($h(x) = 0$).

No entanto, a análise de custo computacional trouxe um dado importante: apesar de o SA encontrar **1 solução** rapidamente, a busca exaustiva pelas **92 soluções distintas** exigiu um grande esforço. Foram necessárias **577 execuções** e quase **meio milhão de iterações** no total, o que prova que mapear completamente o espaço de soluções ótimas, mesmo em um problema conhecido, é a parte mais custosa do processo.

Em suma, as meta-heurísticas são ferramentas poderosas, mas exigem dedicação na calibração para garantir a qualidade e a estabilidade dos resultados em diferentes tipos de problemas.

V. REFERÊNCIAS

[1] Slides do professor
[2] Wikipedia
[3] ChatGPT